

ENSEMBLE LEARNING

AdaBoost Regressor

Turning a sequence of weak learners into one accurate, weighted predictor

● Circle size = sample weight — the visual motif used throughout this deck

What Is Boosting?

A family of ensemble methods that build one strong model from many weak ones — sequentially, not independently.



AdaBoost (Adaptive Boosting) is the original boosting algorithm — the regressor variant extends it to continuous targets.

Core Idea

1

Sequential training

Each new learner is trained after the previous one, on the same data — not in parallel.

2

Error-weighted focus

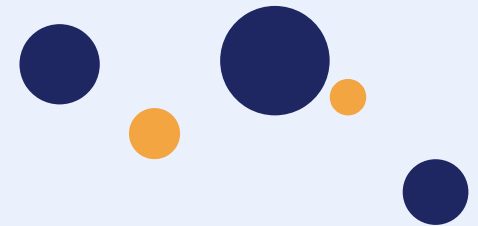
Samples the current ensemble predicts poorly get higher weight for the next learner.

3

Confidence-weighted vote

Each learner's contribution to the final prediction is scaled by how accurate it was.

Weight grows for
missed points



How the Algorithm Runs

1

Initialize weights

Every training sample starts with equal weight, $1/N$.

2

Train a weak learner

Usually a shallow decision tree, fit on the weighted data.

3

Measure weighted error

Compute the loss for each prediction, scaled by sample weight.

4

Compute learner weight

Learners with lower error get a larger say in the final vote.

5

Re-weight the samples

Increase weight on poorly-predicted points for the next round.

6

Repeat & combine

Loop N times, then merge all learners into a weighted prediction.

The Key Formulas

AdaBoost.R2 (Drucker), the formulation scikit-learn uses for regression

1. Sample loss

$$L_i = |y_i - \hat{y}_i| / \max |y_i - \hat{y}_i|$$

Normalized error for each sample under the current learner, scaled 0 to 1.

2. Weighted average loss

$$\bar{e} = \sum w_i \cdot L_i$$

Overall error of this round's learner, weighted by current sample weights.

3. Learner confidence

$$\beta = \bar{e} / (1 - \bar{e})$$

Smaller β means a more accurate learner — it earns a larger vote.

4. Weight update

$$w_i \leftarrow w_i \cdot \beta^{(1 - L_i)}$$

Low-error samples shrink in weight; high-error samples barely shrink.

Prediction & Key Hyperparameters

Weighted Median

Final prediction = the weighted median of all learners' outputs, using $\log(1/\beta)$ as each vote's weight.

Not a simple average — more accurate learners (lower β) get more influence over the final value.

n_estimators

Max number of weak learners
(default 50)

learning_rate

Shrinks each learner's contribution

estimator

Base learner — typically a shallow
DecisionTreeRegressor

loss

linear, square, or exponential re-weighting

Advantages & Limitations

ADVANTAGES

- Often improves on a single weak model with little tuning
- Fewer hyperparameters than most ensemble methods
- Works with any base regressor that accepts sample weights

LIMITATIONS

- Sensitive to noisy data and outliers — they keep gaining weight
- Sequential training means it can't be parallelized like bagging
- Can overfit if `n_estimators` is set too high

When to Reach for AdaBoost Regressor



Small – medium tabular datasets

Where training many shallow trees sequentially is still fast.



A quick accuracy boost

Squeezing more performance out of an existing weak model.



Clean, well-behaved data

Few outliers — since AdaBoost weights noisy points heavily.



Baseline before gradient boosting

A simpler stepping stone toward XGBoost / LightGBM.

Quick Start — scikit-learn

```
from sklearn.ensemble import AdaBoostRegressor
from sklearn.tree import DecisionTreeRegressor

model = AdaBoostRegressor(
    estimator=DecisionTreeRegressor(max_depth=4),
    n_estimators=100,
    learning_rate=0.5,
    loss="linear",
    random_state=42,
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Notes

- A shallow tree (depth 3-5) works better as base learner than a full-depth tree
- Lower learning_rate + more n_estimators often generalizes better
- Always validate with cross-validation before tuning further

Key Takeaways

- 1 Boosts weak learners sequentially, each one correcting the last
- 2 Sample weights grow for poorly-predicted points every round
- 3 Final output is a weighted median, not a plain average
- 4 Simple, low-tuning ensemble — but sensitive to outliers

Thank you